

vLLM专题（四）-故障排除



大模型专题系列 专栏收录该内容

¥49.90
¥99.00

111 篇文章

已订阅

8折续

👑 您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益 >

本文件概述了一些您可以考虑的故障排除策略。如果您认为发现了一个 bug，请首先搜索现有的问题，看看是否已经有人报告。如果没有，请提交一个新问题，并尽可能提供相关的详细信息。

注意

一旦您调试完问题，请记得关闭任何已定义的调试环境变量，或者简单地启动一个新 shell，以避免受残留的调试设置影响。否则，系统可能会因为调试功能未关闭而变得缓慢。

下载模型时卡住

如果模型尚未下载到磁盘，vLLM 将从互联网上下载，这可能需要一些时间，并且取决于您的网络连接。建议首先使用 `huggingface-cli` 下载模型，并将模型的本地路径传递给 vLLM。这样，您可以隔离问题。

1. 从磁盘加载模型时卡住

如果模型很大，从磁盘加载它可能需要较长时间。请注意模型的存储位置。一些集群在节点之间有共享文件系统，例如分布式文件系统或网络文件系统，这可能会比较慢。最好将模型存储在本地磁盘上。此外，检查一下 CPU 内存的使用情况，当模型过大时，它可能占用大量的 CPU 内存，导致操作系统变慢，因为它需要频繁地在磁盘和内存之间交换数据。

注意

为了隔离模型下载和加载问题，您可以使用 `--load-format dummy` 参数跳过加载模型权重。这样，您可以检查模型下载和加载是否是瓶颈。

2. 内存不足

如果模型太大，无法完全加载到单个 GPU 中，您将遇到内存不足（OOM）错误。可以考虑使用张量并行将模型拆分到多个 GPU 上。在这种情况下，每个进程将读取整个模型并将其拆分为多个块，这样磁盘读取时间会更长（与张量并行度成正比）。您可以使用 `examples/offline_inference/save_sharded_state.py` 将模型检查点转换为分片检查点。转换过程可能需要一些时间，但之后您可以更快地加载分片检查点。无论张量并行度的大小如何，模型加载时间应该保持恒定。

3. 启用更多日志记录

如果其他策略无法解决问题，可能是 vLLM 实例卡在了某个地方。您可以使用以下环境变量来帮助调试问题：

`export VLLM_LOGGING_LEVEL=DEBUG` 以启用更多日志记录。

`export CUDA_LAUNCH_BLOCKING=1` 以识别哪个 CUDA 内核导致了问题。

`export NCCL_DEBUG=TRACE` 以启用更多 NCCL 日志记录。

`export VLLM_TRACE_FUNCTION=1` 记录所有函数调用，以便在日志文件中检查，找出哪个函数崩溃或卡住。

4. 网络配置不正确

如果您有复杂的网络配置，vLLM 实例可能无法获取正确的 IP 地址。您可以查找类似于 `DEBUG 06-10 21:32:17 parallel_state.py:88] world_size=8 rank=0 local_rank=0 distributed_init_method=tcp://xxx.xxx.xxx.xxx:54641 backend=nccl` 的日志，IP 地址应该是正确的。如果不是，请使用环境变量 `export VLLM_HOST_IP=<your_ip_address>` 来覆盖 IP 地址。

您可能还需要设置 `export NCCL_SOCKET_IFNAME=<your_network_interface>` 和 `export GLUE_SOCKET_IFNAME=<your_network_interface>` 来指定用于 IP 地址的网络接口。

5. 在 `self.graph.replay()` 附近的错误

如果 vLLM 崩溃，并且错误追踪显示出出错位置在 `vllm/worker/model_runner.py` 的 `self.graph.replay()` 附近，那么这是 CUDAGraph 内的 CUDA 错误。为了识别导致错误的具体 CUDA 操作，您可以在命令行中添加 `--enforce-eager`，或者在 LLM 类中设置 `enforce_eager=True`，以禁用 CUDAGraph 优化并隔离导致错误的具体 CUDA 操作。

6. 硬件/驱动程序不正确

如果无法建立 GPU/CPU 通信，您可以使用以下 Python 脚本，并按照下面的说明确认 GPU/CPU 通信是否正常工作。

```
# Test PyTorch NCCL
import torch
```

```

import torch.distributed as dist
dist.init_process_group(backend="nccl")
local_rank = dist.get_rank() % torch.cuda.device_count()
torch.cuda.set_device(local_rank)
data = torch.FloatTensor([1,] * 128).to("cuda")
dist.all_reduce(data, op=dist.ReduceOp.SUM)
torch.cuda.synchronize()
value = data.mean().item()
world_size = dist.get_world_size()
assert value == world_size, f"Expected {
    world_size}, got {
    value}"

print("PyTorch NCCL is successful!")

# Test PyTorch GLoo
gloo_group = dist.new_group(ranks=list(range(world_size)), backend="gloo")
cpu_data = torch.FloatTensor([1,] * 128)
dist.all_reduce(cpu_data, op=dist.ReduceOp.SUM, group=gloo_group)
value = cpu_data.mean().item()
assert value == world_size, f"Expected {
    world_size}, got {
    value}"

print("PyTorch GLoo is successful!")

if world_size <= 1:
    exit()

# Test vLLM NCCL, with cuda graph
from vllm.distributed.device_communicators.pynccl import PyNcclCommunicator

pynccl = PyNcclCommunicator(group=gloo_group, device=local_rank)
# pynccl is enabled by default for 0.6.5+,
# but for 0.6.4 and below, we need to enable it manually.
# keep the code for backward compatibility when because people
# prefer to read the latest documentation.
pynccl.disabled = False

s = torch.cuda.Stream()
with torch.cuda.stream(s):
    data.fill_(1)
    pynccl.all_reduce(data, stream=s)
    value = data.mean().item()
    assert value == world_size, f"Expected {
        world_size}, got {
        value}"

```

```

print("vLLM NCCL is successful!")

g = torch.cuda.CUDAGraph()
with torch.cuda.graph(cuda_graph=g, stream=s):
    pynccl.all_reduce(data, stream=torch.cuda.current_stream())

data.fill_(1)
g.replay()
torch.cuda.current_stream().synchronize()
value = data.mean().item()
assert value == world_size, f"Expected {
    world_size}, got {
    value}"

print("vLLM NCCL with cuda graph is successful!")

dist.destroy_process_group(gloo_group)
dist.destroy_process_group()

```

如果您在单节点上进行测试，请将 `--nproc-per-node` 调整为您希望使用的 GPU 数量：

```
NCCL_DEBUG=TRACE torchrun --nproc-per-node=<number-of-GPUs> test.py
```

```
NCCL_DEBUG=TRACE torchrun --nproc-per-node=<GPU数量> test.py
```

```
NCCL_DEBUG=TRACE torchrun --nnodes 2 --nproc-per-node=2 --rdzv_backend=c10d --rdzv_endpoint=$MASTER_ADDR test.py
```

如果脚本成功运行，您应该看到消息“sanity check is successful!”。

如果测试脚本卡住或崩溃，通常意味着硬件/驱动程序在某些方面出现了问题。您应该尝试联系系统管理员或硬件供应商以获取进一步的帮助。作为一种常见的解决方法，您可以尝试调节一些 NCCL 环境变量，例如 `export NCCL_P2P_DISABLE=1` 来查看是否有效。有关更多信息，请查阅他们的文档。请仅将这些环境变量作为临时解决方法，因为它们可能会影响系统的性能。最好的解决方案仍然是修复硬件/驱动程序，以便测试脚本能够成功运行。

注意

多节点环境比单节点环境更复杂。如果您看到诸如 `torch.distributed.DistNetworkError` 的错误，可能是网络/DNS 设置不正确。在这种情况下，您可以通过命令行参数手动分配节点排名并指定 IP：

1 在第一个节点上运行：

```
NCCL_DEBUG=TRACE torchrun --nnodes 2 --nproc-per-node=2 --node-rank 0 --master_addr $MASTER_ADDR test.py
```

2 在第二个节点上运行：

```
NCCL_DEBUG=TRACE torchrun --nnodes 2 --nproc-per-node=2 --node-rank 1 --master_addr $MASTER_ADDR test.py
```

根据您的设置调整 `--nproc-per-node`、`--nnodes` 和 `--node-rank`，并确保在不同的节点上执行不同的命令（使用不同的 `--node-rank`）。

7. Python 多处理（multiprocessing）

7.1 RuntimeError 异常

如果在日志中看到类似这样的警告：

```
WARNING 12-11 14:50:37 multiproc_worker_utils.py:281] CUDA was previously
    initialized. We must use the `spawn` multiprocessing start method. Setting
    VLLM_WORKER_MULTIPROC_METHOD to 'spawn'. See
    https://docs.vllm.ai/en/latest/getting_started/troubleshooting.html#python-multiproc
ssing
    for more information.
```

或者看到类似这样的 Python 错误：

```
RuntimeError:
    An attempt has been made to start a new process before the
    current process has finished its bootstrapping phase.

    This probably means that you are not using fork to start your
    child processes and you have forgotten to use the proper idiom
    in the main module:

        if __name__ == '__main__':
            freeze_support()
            ...

    The "freeze_support()" line can be omitted if the program
    is not going to be frozen to produce an executable.

    To fix this issue, refer to the "Safe importing of main module"
    section in https://docs.python.org/3/library/multiprocessing.html
```

那么你必须更新你的 Python 代码，将 vLLM 的使用放在 `if __name__ == '__main__':` 语句块中。例如，改为如下：

```
import vllm

llm = vllm.LLM(...)
```

试试这个：

```
if __name__ == '__main__':
    import vllm

    llm = vllm.LLM(...)
```

7.2 torch.compile 错误

vLLM 严重依赖于 torch.compile 来优化模型以获得更好的性能，这导致了对 torch.compile 功能和 triton 库的依赖。默认情况下，我们使用 torch.compile 来优化模型中的一些函数。在运行 vLLM 之前，您可以通过运行以下脚本来检查 torch.compile 是否按预期工作：

```
import torch

@torch.compile
def f(x):
    # a simple function to test torch.compile
    x = x + 1
    x = x * 2
    x = x.sin()
    return x

x = torch.randn(4, 4).cuda()
print(f(x))
```

如果它从 torch/_inductor 目录中引发错误，通常意味着您有一个与您使用的 PyTorch 版本不兼容的自定义 triton 库。例如，可以查看这个问题。

7.3 模型检查失败

如果你看到类似这样的错误：

```
File "vllm/model_executor/models/registry.py", line xxx, in _raise_for_unsupported
    raise ValueError(
ValueError: Model architectures ['<arch>'] failed to be inspected. Please check the logs
for more details.
```

这意味着 vLLM 导入模型文件失败。通常，这与缺少依赖项或 vLLM 构建中的过时二进制文件有关。请仔细阅读日志，以确定错误的根本原因。

7.4 模型不支持

如果你看到类似的错误：

```
Traceback (most recent call last):
...
  File "vllm/model_executor/models/registry.py", line xxx, in inspect_model_cls
    for arch in architectures:
TypeError: 'NoneType' object is not iterable
```

或者

```
File "vllm/model_executor/models/registry.py", line xxx, in _raise_for_unsupported
    raise ValueError(
ValueError: Model architectures ['<arch>'] are not supported for now. Supported architectures: [...]
```

如果你确定模型在支持的模型列表中，但仍然出现问题，可能是 vLLM 的模型解析存在问题。在这种情况下，请按照以下步骤显式指定模型的 vLLM 实现。

7.5 已知问题

在 v0.5.2、v0.5.3 和 v0.5.3.post1 版本中，由于 zmq 的 bug，可能会导致 vLLM 在某些机器配置下卡住。解决方案是升级到最新版本的 vLLM，以包含修复。

为绕过 NCCL 的 bug，所有 vLLM 进程都会设置环境变量 `NCCL_CUMEM_ENABLE=0`，以禁用 NCCL 的 cuMem 分配器。这个设置不会影响性能，只是带来内存方面的好处。当外部进程要与 vLLM 进程建立 NCCL 连接时，它们也应该设置这个环境变量，否则，环境设置不一致可能导致 NCCL 卡住或崩溃，正如在 RLHF 集成和讨论中所观察到的那样。